

Math Reference

Gabriel Engström

Key formulas used in the system.

1 Probability & Events

1.1 Bucket Probability (from distribution)

$$P(a < X \leq b) = \int_a^b f(x) dx$$

- **In practice:** CDF(b) - CDF(a) for parametric models, numerical integration for KDE.
- Used to match Polymarket bucket structure — each 1°C bucket (e.g. “32°C” = [31.5, 32.5)) is a binary YES/NO contract.

1.2 Gaussian Bucket Probability

$$P(a < X \leq b) = \Phi\left(\frac{b - \mu}{\sigma}\right) - \Phi\left(\frac{a - \mu}{\sigma}\right)$$

- Φ = standard normal CDF
- μ = historical mean
- σ = historical std

2 Bayesian Inference

2.1 Bayes' Theorem

$$P(\theta \mid D) = \frac{P(D \mid \theta) \cdot P(\theta)}{P(D)}$$

- $P(\theta)$ = prior belief
- $P(D \mid \theta)$ = likelihood of data given θ
- $P(D)$ = marginal likelihood (normalizing constant)
- $P(\theta \mid D)$ = posterior (updated belief)

2.2 Sequential Updating

$$P(\theta \mid D_1, D_2) = \frac{P(D_2 \mid \theta) \cdot P(\theta \mid D_1)}{P(D_2)}$$

Each new observation updates the posterior, which becomes the new prior.

3 Kernel Density Estimation (KDE)

3.1 KDE Density Estimate

$$\hat{f}(x) = \frac{1}{n \cdot h} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right)$$

- n = number of data points
- h = bandwidth (controls smoothness)
- K = kernel function (use Gaussian kernel by default)

3.2 Gaussian Kernel

$$K(u) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{u^2}{2}\right)$$

3.3 Bandwidth Selection

Use Silverman’s rule of thumb as default:

$$h = 1.06 \cdot \sigma \cdot n^{-1/5}$$

4 Evaluation — Proper Scoring Rules

4.1 Brier Score (Single Binary Event)

(Reference definition only)

$$BS = \frac{1}{N} \sum_t (f_t - o_t)^2$$

- f_t = predicted probability at time t
- o_t = actual outcome (0 or 1)
- **Range:** $[0, 1]$ — lower is better. Perfect model: $BS = 0$, random model ($f = 0.5$ always): $BS = 0.25$.
- *Note:* This is the textbook single-event form. **Not what we compute** — see Multi-Category Brier Score below, which is what this project actually uses, since the event is a bucket, not a single threshold.

4.2 Multi-Category Brier Score

(Chosen convention — this is what `scoring.py` implements)

Each day has B mutually exclusive 1°C buckets (matching Polymarket’s Hong Kong market structure — see `decisions_log.md`, “Bucket probability formulation”). For day t , bucket b :

- $f_{t,b}$ = model’s predicted probability for bucket b on day t
- $o_{t,b} = 1$ if the actual high landed in bucket b , else 0 — a one-hot vector across buckets for that day (exactly one bucket is correct per day)

$$BS_t = \sum_{b=1}^B (f_{t,b} - o_{t,b})^2$$
$$BS = \frac{1}{T} \sum_{t=1}^T BS_t$$

- T = number of OOS days

- **Range:** $[0, 2]$, not $[0, 1]$ — worst case (100% mass on the wrong bucket) gives $BS_t = 2$. Don't be surprised if the number isn't in $[0, 1]$ like the single-event form above.
- *Note:* This is Brier's original 1950 multi-category formulation. Chosen over the alternative of flattening every $(day, bucket)$ pair into one independent sample and dividing by $T \cdot B$, because a day's bucket probabilities are one distribution over mutually exclusive outcomes, not independent draws — grouping by day before averaging respects that. One trial = one day's full bucket distribution, not one bucket.

4.3 Log Loss (Multi-Category / Categorical Cross-Entropy)

Since $o_{t,b}$ is one-hot, the general cross-entropy sum collapses to just the predicted probability the model assigned to the bucket that actually happened:

$$LL_t = -\log(f_{t,b_t^*}) \quad (\text{where } b_t^* = \text{the actual bucket on day } t)$$

$$LL = \frac{1}{T} \sum_{t=1}^T LL_t$$

- **Lower is better.** Penalizes confident wrong predictions heavily ($\log(0) \rightarrow \infty$).
- **Clip predictions:** $f_{t,b} \in [\epsilon, 1]$ to avoid numerical issues.
- *Note:* This is the categorical analogue of the binary log loss formula — no separate “binary” version is needed here since every event in this project is bucket-based.

4.4 Brier Skill Score (Relative to baseline)

$$BSS = 1 - \frac{BS_{\text{model}}}{BS_{\text{baseline}}}$$

- Both BS_{model} and BS_{baseline} computed via the multi-category convention above (same T , same buckets).
- $BSS > 0$ means model beats baseline. $BSS = 1$ means perfect model.
- **Intended baseline:** Gaussian/KDE climatology (pooled or per-month). Bayesian should beat climatology to justify its extra complexity.

4.5 Expected Calibration Error (ECE)

$$ECE = \sum_{m=1}^M \frac{|B_m|}{N} \cdot |\bar{o}_m - \bar{f}_m|$$

- M = number of bins (typically 10, equal-width over $[0, 1]$)
- B_m = set of samples whose predicted probability falls in bin m
- $|B_m|$ = count of samples in bin m
- N = total number of samples
- $\bar{o}_m$ = observed frequency in bin m (fraction of events that actually occurred)
- $\bar{f}_m$ = mean predicted probability in bin m
- **Range:** $[0, 1]$ — lower is better. $ECE = 0$ means perfect calibration.
- *Note:* Unlike the multi-category Brier score above, ECE uses the **flattened** $(day, bucket)$ pairs as samples — $N = T \cdot B$, not T . There's no “per-day” grouping here.
- **How to use:** Plot $\bar{o}_m$ vs $\bar{f}_m$ (reliability diagram). ECE quantifies the area between that curve and the diagonal. Use alongside Brier score.

5 Mispricing & Edge

5.1 Edge (Gross)

$$\text{edge} = P_{\text{model}}(\text{event}) - P_{\text{market}}(\text{event})$$

- **Positive edge:** model thinks event more likely than market $\rightarrow$ consider buying.
- **Negative edge:** model thinks event less likely than market $\rightarrow$ consider selling.
- Only act when $|\text{edge}| > \text{minimum_edge threshold}$ (e.g. 0.05).

5.2 Effective Edge (After transaction costs)

$$\text{effective_edge} = \text{edge} - \frac{\text{spread}}{2} - \text{fee_rate}$$

- $\text{spread} = \text{ask_price} - \text{bid_price}$ (bid/ask spread on Polymarket)
- $\text{fee_rate} = \text{platform fee}$ (Polymarket charges $\sim 2\%$)
- Only act when both $|\text{edge}| \geq \text{MIN_EDGE}$ and $|\text{effective_edge}| \geq \text{MIN_EFFECTIVE_EDGE}$.
- *Implementation note:* No real per-market historical spread exists for resolved Polymarket markets. `pricing/edge.py:effective_edge()` uses a flat, documented placeholder constant instead of a measured value.

5.3 Transaction Cost in P&L

(As actually implemented, differs from the formula above)

```
# backtest/engine.py --- win:
profit = stake * (1 / price_paid - 1) * (1 - FEE_RATE)
# loss:
profit = -stake
```

Only **FEE_RATE** is deducted from realized profit. **spread** is not — it only gates which trades are eligible to be taken at all (via **effective_edge_flag**), it never reduces a taken trade's actual payoff. The net P&L formula above was the original plan; the actual implementation folds the fee multiplicatively into the win case instead.

5.4 Market-Implied Probability (From decimal odds)

$$P_{\text{market}} = \frac{1}{\text{decimal_odds}}$$

Note: raw market odds include the overround (vig). Adjust:

$$P_{\text{market_adjusted}} = \frac{P_{\text{market}}}{P_{\text{buy}} + P_{\text{sell}}}$$

6 Risk Management

6.1 Kelly Criterion

$$f^* = \frac{b \cdot p - q}{b}$$

- b = net decimal odds (e.g. $b = 2$ means win 2x stake)
- $p = P_{\text{model}}(\text{event})$ = probability of winning
- $q = 1 - p$ = probability of losing
- f^* = fraction of bankroll to stake

Simplified form (when $b = 1$, i.e. even odds):

$$f^* = p - q = 2p - 1$$

Edge form:

$$f^* = \frac{\text{edge}}{b}$$

Constraint note: As actually implemented, `risk/kelly.py` does **not** clamp negative f^* to 0. Since edge is two-sided, a negative f^* is meaningful — it signals betting the **No** side rather than “don’t bet.”

6.2 Fractional Kelly

$$f = \kappa \cdot f^*$$

- $\kappa \in (0, 1)$
- **Typical values:** $\kappa = 0.25$ (quarter Kelly) for conservative sizing. Reduces variance at the cost of slightly lower expected growth.

6.3 Value at Risk (VaR)

$$\text{VaR}_\alpha = -\inf\{x : P(L \leq x) > \alpha\}$$

The loss not exceeded with probability α (e.g. $\alpha = 0.95 \rightarrow 95\%$ VaR). In practice: the α -th percentile of the loss distribution.

6.4 Expected Shortfall (CVaR)

$$\text{ES}_\alpha = E[L \mid L > \text{VaR}_\alpha]$$

Average loss in the worst $(1 - \alpha)$ fraction of scenarios. More informative than VaR — captures tail risk.

6.5 Maximum Drawdown

$$\text{MDD} = \max_{t \in [0, T]} \frac{\text{peak}_t - \text{trough}_t}{\text{peak}_t}$$

- The largest peak-to-trough decline in portfolio value as a fraction of peak.
- *Implementation note:* `risk/metrics.py:rolling_max_dd()` computes the absolute peak-to-current gap, not divided by the peak (it returns a currency-denominated drawdown series, not a fraction). Dividing by the running peak to get a true fractional MDD is not currently implemented.

7 Expected Value

7.1 Expected Value of a Bet

$$EV = p \cdot b - (1 - p) \cdot 1$$

$$EV = p \cdot (b + 1) - 1$$

- b = net odds
- stake = 1 unit
- Act only when $EV > 0$.

8 Calibration

A model is perfectly calibrated if: For all p :

$$P(\text{event} \mid f = p) = p$$

- i.e., among all predictions of 70%, the event occurs 70% of the time.
- **Reliability diagram:** plot mean predicted probability (x-axis) vs observed frequency (y-axis). Perfect calibration = diagonal line.

9 Look-Ahead Bias

This is the most common reason student quant projects fail interviews. A model at time t must only use information available at time t .

- **Wrong:** train on actual observed temperature on day t , then predict day $t + 1$.
- **Right:** train on forecast issued at time t for day $t + 1$, then verify against actual outcome.

```
prediction(t) = f(data available at time t)
outcome(t)    = realized value at time t  # only used for scoring, never for training
```

In walk-forward validation:

```
for each date t:
    train model on data[:t]           # everything strictly before t
    predict P(event at t+horizon)     # using only info available at t
    score against actual outcome[t]   # after the fact
```

Never let future outcomes leak into the training window.